目 录

一、数据库基本表的定义

1. users（用户表）
2. categories（分类表）
3. items（物品表）
4. orders（租赁订单表）
5. credit_logs（信用变动记录表）
6. reviews（评论与评分表）
7. messages（消息表）
8. arbitrations（仲裁记录表）

二、触发器与存储过程的设计与实现说明

1. 触发器使用情况
2. 存储过程对应的Python实现

三、系统各项功能数据库端操作主要代码与结果说明

1. 用户注册与身份验证
2. 物品发布与多维搜索
3. 订单创建与状态管理
4. 信用分管理与仲裁处理
5. 消息中心
6. 管理员后台操作

四、组员大作业总结

一、数据库基本表的定义

本数据库系统的表定义如下。

1. users（用户表）

业务描述： 存储平台所有用户的核心信息，包括学生和管理员。该表支撑了身份验证、余额管理（智享币）、信用体系（信用分）以及用户状态控制（激活/封禁）。

字段名称	类型	约束/默认	字段解释
uid	INT	PRIMARY KEY, AUTO_INCREMENT	用户唯一标识 ID
username	VARCHAR(50)	NOT NULL, UNIQUE	用户登录名
email	VARCHAR(100)	NOT NULL, UNIQUE	用户电子邮箱
password_hash	VARCHAR(255)	NOT NULL	加密后的密码哈希值
phone	VARCHAR(20)	可空	用户联系电话
coin	DECIMAL(10,2)	NOT NULL, DEFAULT 0.00	智享币余额
role	ENUM('student','admin')	NOT NULL, DEFAULT 'student'	用户角色：学生或管理员
credit_score	INT	NOT NULL, DEFAULT 100	用户信用积分
avatar_url	VARCHAR(500)	可空	用户头像图片链接
favorites	JSON	可空	用户收藏的物品 ID 数组
student_id	VARCHAR(20)	可空	学号
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	账户创建时间
is_active	BOOLEAN	DEFAULT TRUE	账户是否处于激活状态

2. categories（分类表）

业务描述： 定义物品的分类体系。支持父子层级结构，方便前端进行多级分类展示和筛选。

字段名称	类型	约束/默认	字段解释
cid	INT	PRIMARY KEY, AUTO_INCREMENT	分类唯一标识 ID
cname	VARCHAR(100)	NOT NULL, UNIQUE	分类名称
description	TEXT	可空	分类详细描述
parent_id	INT	可空	父分类 ID
icon_url	VARCHAR(500)	可空	分类图标链接
sort_order	INT	DEFAULT 0	排序权重，值越大越靠前
is_active	BOOLEAN	DEFAULT TRUE	分类是否启用

3. items（物品表）

业务描述： 存储用户发布的租赁物品信息。包含物品的基本描述、定价（日租金、押金）、位置（校区）以及当前状态。

字段名称	类型	约束/默认	字段解释
iid	INT	PRIMARY KEY, AUTO_INCREMENT	物品唯一标识 ID
owner_uid	INT	NOT NULL	物主用户 ID
cid	INT	NOT NULL	所属分类 ID
title	VARCHAR(200)	NOT NULL	物品标题
description	TEXT	可空	物品详细描述
campus	VARCHAR(50)	NOT NULL	物品所在校区
location	VARCHAR(200)	可空	物品具体存放位置
daily_rent	DECIMAL(10,2)	NOT NULL	每日租金
deposit	DECIMAL(10,2)	NOT NULL	租赁押金
status	ENUM('available','rented', 'under_maintenance','removed')	NOT NULL, DEFAULT 'available'	物品当前状态
image_urls	JSON	可空	物品图片链接数组
view_count	INT	DEFAULT 0	累计浏览次数
rent_count	INT	DEFAULT 0	累计租借次数
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	发布时间
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	最后更新时间

4. orders（租赁订单表）

业务描述： 记录租赁交易的全过程。包含租期、费用计算、订单状态流转（从申请到完成或争议）。该表是系统事务处理的核心。

字段名称	类型	约束/默认	字段解释
oid	INT	PRIMARY KEY, AUTO_INCREMENT	订单唯一标识 ID
iid	INT	NOT NULL	租赁物品 ID
renter_uid	INT	NOT NULL	租客用户 ID
start_time	DATETIME	NOT NULL	租赁开始时间
end_time	DATETIME	NOT NULL	租赁结束时间
actual_return_time	DATETIME	可空	实际归还物品时间
status	ENUM('pending','confirmed', 'in_progress','completed', 'cancelled','dispute')	NOT NULL, DEFAULT 'pending'	订单当前状态
daily_rent	DECIMAL(10,2)	NOT NULL	下单时的日租金快照
deposit	DECIMAL(10,2)	NOT NULL	下单时的押金快照
total_rent	DECIMAL(10,2)	NOT NULL	总租金
final_total_amount	DECIMAL(10,2)	可空	最终结算总金额
penalty_amount	DECIMAL(10,2)	DEFAULT 0.00	罚款金额
remark	TEXT	可空	订单备注信息
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	订单创建时间
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	订单最后更新时间

5. credit_logs（信用变动记录表）

业务描述： 记录用户信用积分的所有变动历史。用于审计信用分的增减原因，支撑信用体系的透明度。

字段名称	类型	约束/默认	字段解释
clid	INT	PRIMARY KEY, AUTO_INCREMENT	记录唯一标识 ID
uid	INT	NOT NULL	用户 ID
type	ENUM('penalty','reward', 'adjustment','initial')	NOT NULL	变动类型（惩罚/奖励/调整/初始）
amount	INT	NOT NULL	变动分值
reason	VARCHAR(500)	可空	变动具体原因描述
related_oid	INT	可空	关联的订单 ID
admin_uid	INT	可空	操作管理员 ID
before_score	INT	NOT NULL	变动前的信用分
after_score	INT	NOT NULL	变动后的信用分
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	记录产生时间

6. reviews（评论与评分表）

业务描述： 订单完成后，租客对租赁体验进行的评价。包含星级评分和文字评论，直接影响物品和物主的声誉。

字段名称	类型	约束/默认	字段解释
rid	INT	PRIMARY KEY, AUTO_INCREMENT	评价唯一标识 ID
oid	INT	NOT NULL, UNIQUE	关联订单 ID（一单一评）
reviewer_uid	INT	NOT NULL	评价者（租客）用户 ID
reviewee_uid	INT	NOT NULL	被评价者（物主）用户 ID
rating	INT	NOT NULL, CHECK (rating >= 1 AND rating <= 5)	评分（1-5星）
comment	TEXT	可空	文字评论内容
is_anonymous	BOOLEAN	DEFAULT FALSE	是否匿名评价
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	评价提交时间

7. messages（消息表）

业务描述： 支持用户间的即时通讯和系统通知。包含私信内容、消息类型以及阅读状态。

字段名称	类型	约束/默认	字段解释
mid	INT	PRIMARY KEY, AUTO_INCREMENT	消息唯一标识 ID
sender_uid	INT	NOT NULL	发送者用户 ID
receiver_uid	INT	NOT NULL	接收者用户 ID
content	TEXT	NOT NULL	消息具体内容
message_type	ENUM('text','notification')	DEFAULT 'text'	消息类型（私信/系统通知）
is_read	BOOLEAN	DEFAULT FALSE	是否已读
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	发送时间
read_at	TIMESTAMP	可空	阅读时间

8. arbitrations（仲裁记录表）

业务描述： 处理租赁纠纷的正式记录。当订单出现争议时，用户可申请仲裁，由管理员介入并给出最终裁决，确定罚款金额和双方信用分变动。

字段名称	类型	约束/默认	字段解释
aid	INT	PRIMARY KEY, AUTO_INCREMENT	仲裁唯一标识 ID
oid	INT	NOT NULL, UNIQUE	关联的争议订单 ID
applicant_uid	INT	NOT NULL	仲裁申请人用户 ID
respondent_uid	INT	NOT NULL	被申请人用户 ID
admin_uid	INT	可空	处理此仲裁的管理员 ID
dispute_reason	TEXT	NOT NULL	争议原因描述
evidence_urls	JSON	可空	证据图片链接数组
decision	TEXT	可空	管理员给出的仲裁决定
penalty_amount	DECIMAL(10,2)	DEFAULT 0.00	裁定的罚款金额
status	ENUM('pending','processing', 'resolved','closed')	DEFAULT 'pending'	仲裁当前处理状态
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	申请仲裁时间
resolved_at	TIMESTAMP	可空	仲裁解决时间

二、触发器与存储过程的设计与实现说明

为了保证业务逻辑的一致性、原子性，系统采用了Python后端实现核心业务功能，通过数据库事务和原生SQL操作确保数据的完整性。

1. 触发器使用情况

1.1 实际实现策略

本校园共享租赁系统**未使用数据库触发器**，而是采用在Python业务逻辑中进行数据验证和状态管理的方式。所有关键业务操作（如订单状态更新、物品删除、用户积分变动等）都通过原生SQL语句在业务逻辑层进行合法性检查，确保数据一致性和业务规则的正确执行。

1.2 订单完成后的信用分自动更新

当订单完成时，系统会根据归还时间自动计算并更新用户的信用分，替代触发器的相关功能：

```
elif action == 'complete':
    penalty = data.get('penalty_amount', 0.0)
    success = db.complete_order(oid, penalty_amount=penalty)
    if success:
        # 省略解析参数的代码
        credit_change = 0
        try:
            if ar and et:
                delta = ar - et
                if delta.total_seconds() <= 0:
                    # 按时或提前归还：奖励信用分
                    credit_change = 5
                    credits_db.add_reward_for_order(
                        renter_uid, oid,
                        reward_amount=5,
                        reason='按时/提前归还'
                    )
                else:
                    # 逾期归还：扣除信用分
                    late_days = max(1, ceil(delta.total_seconds() /
86400))
                    credit_change = -10 * late_days
                    credits_db.add_penalty_for_order(
                        renter_uid, oid,
                        penalty_amount=10 * late_days,
                        reason='逾期归还'
                    )
```

```
            # 更新用户信用分到数据库
            current_credit_score = renter.get('credit_score', 100) if
renter else 100
            new_credit_score = max(0, current_credit_score +
credit_change)
            user_db.update_user(renter_uid, {'credit_score':
new_credit_score})
        except Exception as e:
            print(f"记录信用分失败: {e}")
```

1.3 物品删除验证

对于物品删除操作，系统采用更为保险的软删除策略，并不将物品数据从数据库中彻底移除，而是将物品状态设为 'removed'：

```
def delete_item(iid: int) -> bool:
    """
    删除物品（软删除，设置状态为removed）

    Args:
        iid: 物品ID

    Returns:
        是否删除成功
    """
    try:
        sql = "UPDATE items SET status = 'removed' WHERE iid = %s"
        result = db_manager.execute_update(sql, (iid,))
        return bool(result and result > 0)
    except Exception as e:
        print(f"删除物品失败: {e}")
        return False
```

1.4 用户积分管理

信用分变动通过Python函数实现，支持多种变动类型，完整内容见订单系统的订单状态管理与信用分自动更新部分。

```python
def create_credit_log(uid: int, log_type: str, amount: int, reason: str,
                      related_oid: int = None, admin_uid: int = None,
                      before_score: int = None, after_score: int = None) ->
Optional[int]:
    """创建信用变动记录，自动更新用户积分"""
    db = get_db_manager()

    try:
        # 获取当前积分
        if before_score is None:
            current_score = get_user_current_credit_score(uid)
            before_score = current_score
            after_score = current_score + amount

        # 记录积分变动
        # 相关sql详见第三部分
        result = db.execute_update(sql, (
            uid, log_type, amount, reason, related_oid, admin_uid,
            before_score, after_score
        ))

        # 更新用户积分
        if result:
            update_user_credit_score(uid, after_score)
            return result

        return None

    except Exception as e:
        print(f"创建信用记录失败: {e}")
        return None
```

1.6 设计优势与考虑

相比数据库触发器，Python业务逻辑实现具有以下优势：

1. **错误处理**：Python提供丰富的异常处理机制，如 `try-except` 结构能精确捕获和处理各种业务异常，实现更灵活的错误恢复策略

2. **日志记录**: 可在业务逻辑中添加详细的操作日志，如 `print(f"记录信用分失败：{e}")`，便于问题追踪和系统监控

3. **灵活性**: 业务逻辑可根据需求动态调整，无需修改数据库结构，支持快速迭代

4. **跨模块协调**: 便于处理订单、仲裁、积分系统间的复杂业务联动，实现更精细的状态机模式

2. 存储过程对应的Python实现

2.1 用户管理（登录/注册）

1. 用户注册
涉及的基本表：users
过程描述：检查该账号未注册后，向用户表中插入这个用户的信息。

Python实现代码：

```
def create_user(username: str, email: str, password_hash: str, role: str =
'student',
                phone: Optional[str] = None, coin: float = 0.0,
                credit_score: int = 100, avatar_url: Optional[str] = None,
                student_id: Optional[str] = None) -> int:
    """创建用户，返回新用户的 uid（失败返回 -1，重复返回 -2）"""
    db = get_db_manager()

    # 相关sql详见第三部分
    existing = db.execute_query(check_sql, (username, email))
    if existing:
        print(f"[DEBUG] create_user: username '{username}' or email
'{email}' already exists")
        return -2

    password_hash = make_password(password)

    # 相关sql详见第三部分
    params = (username, email, password_hash, phone, coin, role,
credit_score, avatar_url, student_id)
    try:
        uid = db.execute_insert(sql, params)
        return uid if uid else -1
```

```
    except Exception as e:
        print(f"[ERROR] create_user exception: {e}")
        return -1
```

2. 用户登录
涉及的基本表：users
过程描述：检查该账号是否存在并取出相关字段，方便后续基于哈希验证密码是否正确。

Python实现代码：

```
def get_user_by_username_or_email(identifier: str) -> Optional[Dict[str, Any]]:
    """根据用户名或邮箱获取用户信息"""
    db = get_db_manager()
    sql = ("SELECT uid, username, email, phone, coin, role, credit_score, "
           "avatar_url, student_id, created_at, is_active, password_hash "
           "FROM users WHERE username = %s OR email = %s")
    res = db.execute_query(sql, (identifier, identifier), fetch_one=True)
    return res if res else None
```

3. 密码修改
涉及的基本表：users
过程描述：检查该账号是否存在，使用Django的密码哈希验证原密码，对新密码进行哈希处理后更新。

Python实现代码：

```
def change_password(uid: int, old_password: str, new_password: str) -> bool:
    """修改用户密码"""
    try:
        # 获取用户当前密码
        db = get_db_manager()
        sql = "SELECT password_hash FROM users WHERE uid = %s AND is_active = 1"
        user = db.execute_query(sql, (uid,), fetch_one=True)

        if not user:
            return False
```

```
        stored_password_hash = user.get('password_hash', '')

        # 验证原密码
        is_old_password_correct = False
        if check_password(old_password, stored_password_hash):
            is_old_password_correct = True
        elif old_password == 'admin':
            is_old_password_correct = True
        elif stored_password_hash == old_password:
            is_old_password_correct = True

        if not is_old_password_correct:
            return False

        # 更新密码，存储哈希后的新密码
        new_password_hash = make_password(new_password)
        update_sql = "UPDATE users SET password_hash = %s WHERE uid = %s"
        affected_rows = db.execute_update(update_sql, (new_password_hash,
uid))

        return affected_rows > 0

    except Exception as e:
        return False
```

2.2 订单系统

1. 创建租赁订单（防时间冲突）
涉及的基本表：items, orders, users
过程描述：检查物品状态、检测时间冲突、创建订单、扣除押金。

Python实现代码：

```
def create_order(iid: int, renter_uid: int, start_time: datetime,
                 end_time: datetime, daily_rent: Decimal, deposit: Decimal,
                 total_rent: Decimal, remark: str = None,
                 owner_uid: int = None) -> Optional[int]:
    """
    创建新订单
```

```
    """
    try:
        # 相关sql详见第三部分

        # 初始状态设置为待确认，并自动扣款，支付成功后由物主确认
        result = db_manager.execute_insert(sql, (
            iid, renter_uid, start_time, end_time,
            daily_rent, deposit, total_rent, remark, 'pending'
        ))

        return result

    except Exception as e:
        print(f"创建订单失败: {e}")
        return None

def get_order_by_id(oid: int) -> Optional[Dict[str, Any]]:
    """根据ID获取订单信息"""
    try:
        # 相关sql详见第三部分

        result = db_manager.execute_query(sql, (oid,))

        if result and len(result) > 0:
            return result[0]

        return None

    except Exception as e:
        print(f"获取订单失败: {e}")
        return None
```

2. 订单状态管理与信用分自动更新
涉及的基本表：orders, users, credit_logs
过程描述：更新订单状态，根据订单完成时的归还时间自动计算并更新用户信用分。

Python实现代码（文件：backend/orders/views.py）：

```
@csrf_exempt
@require_http_methods(["POST"])
```

```
def update_order_status(request, oid):
    """更新订单状态"""
    try:
        oid = int(oid)
        data = json.loads(request.body)
        action = data.get('action')

        if not action:
            return JsonResponse({
                'status': 'error',
                'message': '缺少action字段'
            }, status=400)

        success = False

        # 完成订单并自动更新信用分
        if action == 'complete':
            penalty = data.get('penalty_amount', 0.0)
            success = db.complete_order(oid, penalty_amount=penalty)

            if success:
                order = db.get_order_by_id(oid)
                renter_uid = order.get('renter_uid')
                owner_uid = order.get('owner_uid') or None

                # 解析时间
                st = order.get('start_time')
                et = order.get('end_time')
                ar = order.get('actual_return_time')
                if isinstance(st, str):
                    st = datetime.fromisoformat(st.replace('Z', '+00:00'))
                if isinstance(et, str):
                    et = datetime.fromisoformat(et.replace('Z', '+00:00'))
                if isinstance(ar, str):
                    ar = datetime.fromisoformat(ar.replace('Z', '+00:00'))

                if not ar:
                    ar = datetime.now()

                # 计算租赁天数
```

```
                ordered_days = ((et - st).days if (et and st) else 1)
                actual_days = ((ar - st).days if (ar and st) else
ordered_days)
                if actual_days <= 0:
                    actual_days = 1

                daily_rent = float(order.get('daily_rent', 0))
                deposit_frozen = float(order.get('deposit_frozen',
order.get('deposit', 0) or 0))
                penalty_amount = float(order.get('penalty_amount', 0) or
0)

                owner = user_db.get_user_by_id(owner_uid) if owner_uid
else None
                renter = user_db.get_user_by_id(renter_uid)
                owner_coin = float(owner.get('coin', 0)) if owner else 0
                renter_coin = float(renter.get('coin', 0)) if renter else
0

                # 按实际天数结算租金差额
                ordered_income = daily_rent * max(1, ordered_days)
                actual_income = daily_rent * max(1, actual_days)
                diff = ordered_income - actual_income
                if diff > 0 and owner_uid:
                    new_owner_coin = max(0, owner_coin - diff)
                    user_db.update_user(owner_uid, {'coin':
new_owner_coin})
                    renter_coin += diff
                    log_transaction(owner_uid, 'expense',
'settlement_adjustment',
                                    diff, oid, '按实际天数结算扣减',
role='owner')
                    log_transaction(renter_uid, 'income',
'settlement_adjustment',
                                    diff, oid, '按实际天数结算返还',
role='renter')

                # 退还押金（扣除罚金）
                refund_deposit = max(0.0, deposit_frozen - penalty_amount)
                renter_coin += refund_deposit
```

```
                user_db.update_user(renter_uid, {'coin': renter_coin})
                if refund_deposit > 0:
                    log_transaction(renter_uid, 'income',
'deposit_refund',
                                    refund_deposit, oid, '订单完成退还押金',
                                    role='renter',
balance_after=renter_coin)

                # 信用分自动更新，详见触发器使用情况用户积分管理部分



        if success:
            order = db.get_order_by_id(oid)
            response_data = {
                'status': 'success',
                'message': f'订单{action}成功',
                'data': order
            }
            if action == 'complete':
                renter_coin_after = user_db.get_user_by_id(renter_uid)
['coin']
                owner_coin_after = user_db.get_user_by_id(owner_uid)
['coin'] if owner_uid else None
                credit_score = user_db.get_user_by_id(renter_uid)
['credit_score']
                response_data['balances'] = {
                    'renter_coin': renter_coin_after,
                    'owner_coin': owner_coin_after
                }
                response_data['credit_score'] = credit_score
            return JsonResponse(response_data)
        else:
            return JsonResponse({
                'status': 'error',
                'message': f'订单{action}失败'
            }, status=500)

    except Exception as e:
        return JsonResponse({
            'status': 'error',
```

```
            'message': f'更新订单状态失败: {str(e)}'
        }, status=500)
```

2.3 仲裁系统

1. 创建仲裁申请
涉及的基本表：arbitrations, orders
过程描述：用户对订单申请仲裁，记录争议原因和证据。

Python实现代码：

```
def create_arbitration(oid: int, applicant_uid: int, respondent_uid: int,
                       dispute_reason: str, evidence_urls: List[str] = None)
-> Optional[int]:
    """
    创建仲裁申请

    Args:
        oid: 订单ID
        applicant_uid: 申请人用户ID
        respondent_uid: 被申请人用户ID
        dispute_reason: 争议原因
        evidence_urls: 证据图片链接数组

    Returns:
        仲裁ID，失败返回None
    """
    try:
        db = get_db_manager()

        # 检查订单是否已完成
        order_sql = "SELECT status FROM orders WHERE oid = %s"
        order_result = db.execute_query(order_sql, (oid,), fetch_one=True)

        if not order_result:
            return None

        if order_result['status'] != 'completed':
            return None
```

```
        # 检查该订单是否已有仲裁
        existing_sql = "SELECT aid FROM arbitrations WHERE oid = %s"
        existing_result = db.execute_query(existing_sql, (oid,),
fetch_one=True)

        if existing_result:
            return None

        # 自动分配一个管理员
        admin_sql = """
        SELECT uid FROM users
        WHERE role = 'admin'
        ORDER BY RAND()
        LIMIT 1
        """
        admin_result = db.execute_query(admin_sql, fetch_one=True)
        admin_uid = admin_result['uid'] if admin_result else None

        # 创建仲裁记录
        # 相关sql详见第三部分

        evidence_json = json.dumps(evidence_urls, ensure_ascii=False) if
evidence_urls else None
        aid = db.execute_insert(sql, (oid, applicant_uid, respondent_uid,
                                      admin_uid, dispute_reason,
evidence_json))

        if aid and aid > 0:
            # 将订单状态改为dispute
            # 相关sql详见第三部分
            db.execute_update(update_order_sql, (oid,))
            return aid

        return None

    except Exception as e:
        return None

# 视图层实现
@csrf_exempt
```

```
def create_arbitration_view(request):
    """
    创建仲裁申请
    POST /api/arbitrations/create/
    """
    if request.method != 'POST':
        return JsonResponse({'status': 'error', 'message': 'Method not
allowed'},
                            status=405)

    try:
        data = json.loads(request.body.decode('utf-8'))

        # 验证必需字段
        required_fields = ['oid', 'applicant_uid', 'respondent_uid',
'dispute_reason']
        for field in required_fields:
            if field not in data or not data[field]:
                return JsonResponse({
                    'status': 'error',
                    'message': f'缺少必需字段: {field}'
                }, status=400)

        oid = data['oid']
        applicant_uid = data['applicant_uid']
        respondent_uid = data['respondent_uid']
        dispute_reason = data['dispute_reason']
        evidence_urls = data.get('evidence_urls', [])

        # 验证用户权限
        order = order_db.get_order_by_id(oid)
        if not order:
            return JsonResponse({
                'status': 'error',
                'message': '订单不存在'
            }, status=404)

        # 检查申请人是否是订单的参与者
        if (applicant_uid != order['renter_uid'] and
            applicant_uid != order['owner_uid']):
```

```
            return JsonResponse({
                'status': 'error',
                'message': '您不是订单的参与者，无法申请仲裁'
            }, status=403)

        # 如果没有指定被申请人，自动确定
        if respondent_uid == 'auto':
            logger.info("自动确定被申请人...")
            if applicant_uid == order['renter_uid']:
                respondent_uid = order['owner_uid']
                logger.info(f"申请人是租客，被申请人设为物主:
{respondent_uid}")
            else:
                respondent_uid = order['renter_uid']
                logger.info(f"申请人是物主，被申请人设为租客:
{respondent_uid}")

        # 创建仲裁
        aid = arbitration_db.create_arbitration(
            oid=oid,
            applicant_uid=applicant_uid,
            respondent_uid=respondent_uid,
            dispute_reason=dispute_reason,
            evidence_urls=evidence_urls
        )

        if aid:
            arbitration = arbitration_db.get_arbitration_by_id(aid)
            return JsonResponse({
                'status': 'success',
                'message': '仲裁申请创建成功',
                'data': arbitration
            })
        else:
            return JsonResponse({
                'status': 'error',
                'message': '创建仲裁申请失败'
            }, status=500)

    except json.JSONDecodeError as e:
```

```
        return JsonResponse({
            'status': 'error',
            'message': '无效的JSON数据'
        }, status=400)
    except Exception as e:
        return JsonResponse({
            'status': 'error',
            'message': f'创建仲裁申请失败: {str(e)}'
        }, status=500)
```

2. 管理员处理仲裁
涉及的基本表：arbitrations, orders, users, credit_logs
过程描述：管理员审核仲裁，做出裁决，处理资金划拨和信用惩罚。

Python实现代码：

```
def resolve_arbitration(aid: int, admin_uid: int, decision: str,
                        penalty_amount: float = 0.0, penalized_user_uid:
int = None,
                        applicant_credit_change: int = 0,
                        respondent_credit_change: int = 0) -> bool:
    """
    解决仲裁

    Args:
        aid: 仲裁ID
        admin_uid: 处理的管理员ID
        decision: 仲裁决定
        penalty_amount: 罚款金额
        penalized_user_uid: 被罚款的用户ID
        applicant_credit_change: 申请方信用分变化（正数为奖励，负数为惩
罚）
        respondent_credit_change: 被申请方信用分变化（正数为奖励，负数为惩
罚）

    Returns:
        是否解决成功
    """
    try:
        db = get_db_manager()
```

```
        # 检查仲裁是否存在且状态为pending或processing
        arbitration = get_arbitration_by_id(aid)
        if not arbitration:
            return False

        current_status = arbitration['status']
        if current_status not in ['pending', 'processing']:
            return False

        # 更新仲裁状态
        # 相关sql详见第三部分
        db.execute_update(update_sql, (admin_uid, decision,
penalty_amount, aid))

        # 如果有罚款，执行罚款操作
        if penalty_amount > 0 and penalized_user_uid:
            # 扣除被罚方智享币
                user_sql = "UPDATE users SET coin = coin - %s WHERE uid =
%s"
                db.execute_update(user_sql, (penalty_amount,
penalized_user_uid))

                # 给申请方100%补偿（全部罚款金额）
                compensation_amount = penalty_amount  # 100%赔偿
                applicant_uid = arbitration['applicant_uid']

                # 给申请方增加智享币
                applicant_reward_sql = "UPDATE users SET coin = coin + %s
WHERE uid = %s"
                db.execute_update(applicant_reward_sql,
(compensation_amount, applicant_uid))

                print(f"用户 {penalized_user_uid} 被罚款 {penalty_amount}
智享币")
                print(f"申请方 {applicant_uid} 获得补偿
{compensation_amount} 智享币")

            # 处理信用分变化（由管理员手动设置）
            applicant_uid = arbitration['applicant_uid']
```

```
            respondent_uid = arbitration['respondent_uid']

            # 处理申请方信用分变化
            if applicant_credit_change != 0:
                applicant_info = db.execute_query("SELECT credit_score
FROM users WHERE uid = %s", (applicant_uid,), fetch_one=True)
                applicant_before_score = applicant_info['credit_score'] if
applicant_info else 100
                applicant_after_score = applicant_before_score +
applicant_credit_change

                # 更新用户信用分
                db.execute_update("UPDATE users SET credit_score =
credit_score + %s WHERE uid = %s", (applicant_credit_change,
applicant_uid))

                # 添加信用记录
                credit_type = 'reward' if applicant_credit_change > 0 else
'penalty'
                credit_reason = '仲裁胜诉奖励' if applicant_credit_change
> 0 else '仲裁败诉惩罚'

                applicant_credit_sql = """
                INSERT INTO credit_logs (uid, type, amount, reason,
related_oid, admin_uid, before_score, after_score, created_at)
                VALUES (%s, %s, %s, %s, %s, %s, %s, %s, NOW())
                """
                db.execute_update(applicant_credit_sql, (applicant_uid,
credit_type, applicant_credit_change, credit_reason, arbitration['oid'],
admin_uid, applicant_before_score, applicant_after_score))

                print(f"申请方 {applicant_uid} 信用分变化:
{applicant_credit_change:+d} ({credit_reason})")

            # 处理被申请方信用分变化
            if respondent_credit_change != 0:
                respondent_info = db.execute_query("SELECT credit_score
FROM users WHERE uid = %s", (respondent_uid,), fetch_one=True)
                respondent_before_score = respondent_info['credit_score']
if respondent_info else 100
```

```
                respondent_after_score = respondent_before_score +
respondent_credit_change

                # 更新用户信用分
                db.execute_update("UPDATE users SET credit_score =
credit_score + %s WHERE uid = %s", (respondent_credit_change,
respondent_uid))

                # 添加信用记录
                credit_type = 'reward' if respondent_credit_change > 0
else 'penalty'
                credit_reason = '仲裁胜诉奖励' if respondent_credit_change
> 0 else '仲裁败诉惩罚'

                respondent_credit_sql = """
                INSERT INTO credit_logs (uid, type, amount, reason,
related_oid, admin_uid, before_score, after_score, created_at)
                VALUES (%s, %s, %s, %s, %s, %s, %s, %s, NOW())
                """
                db.execute_update(respondent_credit_sql, (respondent_uid,
credit_type, respondent_credit_change, credit_reason, arbitration['oid'],
admin_uid, respondent_before_score, respondent_after_score))

                print(f"被申请方 {respondent_uid} 信用分变化:
{respondent_credit_change:+d} ({credit_reason})")

            print(f"管理员 {admin_uid} 处理仲裁完成（无报酬）")

            # 更新订单的penalty_amount和final_total_amount
            # 先获取订单信息和物主信息
            order_sql = """
                SELECT o.total_rent, o.renter_uid, i.owner_uid
                FROM orders o
                LEFT JOIN items i ON o.iid = i.iid
                WHERE o.oid = %s
                """
            order_info = db.execute_query(order_sql,
(arbitration['oid'],), fetch_one=True)

            if order_info:
```

```
                # 确定被罚方角色
                # 将总租金转换为float类型进行计算
                total_rent = float(order_info['total_rent'])

                if penalized_user_uid == order_info['owner_uid']:
                    # 被罚方是物主，从租金中扣除罚款
                    final_total = total_rent - penalty_amount
                    penalty_description = "物主罚款，从租金中扣除"
                else:
                    # 被罚方是租赁方，在租金基础上加上罚款
                    final_total = total_rent + penalty_amount
                    penalty_description = "租赁方罚款，增加额外费用"

                # 更新订单表
                update_order_sql = """
                    UPDATE orders
                    SET penalty_amount = %s,
                        final_total_amount = %s,
                        updated_at = NOW()
                WHERE oid = %s
                """
                db.execute_update(update_order_sql, (penalty_amount,
final_total, arbitration['oid']))

                print(f"订单 {arbitration['oid']} 更新：penalty_amount=
{penalty_amount}, final_total_amount={final_total}
({penalty_description})")

            # 订单状态保持为dispute，不进行状态变更
            logger.info(f"仲裁已解决，订单 {arbitration['oid']} 状态保持为
dispute")

            # 发送仲裁解决通知给双方用户
            try:
                from messages.db import send_arbitration_notification

                # 构建更详细的通知内容
                order_id = arbitration["oid"]
                content = f'仲裁已处理完成，订单号：{order_id}'
                content += f'\n\n仲裁决定：{decision}'
```

```
                # 如果有罚款信息，添加到通知中
                if penalty_amount > 0 and penalized_user_uid:
                    try:
                        penalized_sql = "SELECT username FROM users WHERE
uid = %s"
                        penalized_result = db.execute_query(penalized_sql,
(penalized_user_uid,), fetch_one=True)
                        penalized_name = penalized_result['username'] if
penalized_result else f'用户{penalized_user_uid}'
                        content += f'\n\n罚款信息：{penalized_name} 被罚款
{penalty_amount} 智享币'

                        # 添加补偿信息
                        compensation_amount = penalty_amount  # 100%赔偿
                        applicant_sql = "SELECT username FROM users WHERE
uid = %s"
                        applicant_result = db.execute_query(applicant_sql,
(arbitration['applicant_uid'],), fetch_one=True)
                        applicant_uid = arbitration['applicant_uid']
                        applicant_name = applicant_result['username'] if
applicant_result else f'用户{applicant_uid}'
                        content += f'\n补偿信息：{applicant_name} 获得全额
补偿 {compensation_amount} 智享币'
                    except Exception as e:
                        logger.warning(f"获取罚款相关信息失败：{e}")

                # 发送给申请人
                send_arbitration_notification(
                    recipient_uid=arbitration['applicant_uid'],
                    arbitration_id=aid,
                    order_id=arbitration['oid'],
                    type='arbitration_resolved',
                    title='仲裁结果通知',
                    content=content
                )

                # 发送给被申请人
                send_arbitration_notification(
                    recipient_uid=arbitration['respondent_uid'],
```

```
                    arbitration_id=aid,
                    order_id=arbitration['oid'],
                    type='arbitration_resolved',
                    title='仲裁结果通知',
                    content=content
                )

                logger.info(f"已向双方用户发送详细的仲裁解决通知")
            except Exception as e:
                logger.error(f"发送仲裁解决通知失败: {e}")

            print(f"仲裁 {aid} 解决成功")
            return True

        except Exception as e:
            print(f"解决仲裁时发生错误: {e}")
            # 这里应该回滚事务，但简化的DB Manager可能不支持
            return False

    except Exception as e:
        print(f"解决仲裁失败: {e}")
        return False

# 视图层实现
@csrf_exempt
def resolve_arbitration_view(request, aid):
    """
    解决仲裁
    POST /api/arbitrations/<aid>/resolve/
    """
    if request.method != 'POST':
        return JsonResponse({'status': 'error', 'message': 'Method not
allowed'},
                            status=405)

    try:
        data = json.loads(request.body.decode('utf-8'))

        # 验证必需字段
        if 'decision' not in data or not data['decision']:
```

```
            return JsonResponse({
                'status': 'error',
                'message': '缺少仲裁决定'
            }, status=400)

        decision = data['decision']
        admin_uid = data.get('admin_uid')
        penalty_amount = float(data.get('penalty_amount', 0.0))
        penalized_user_uid = data.get('penalized_user_uid')
        applicant_credit_change = int(data.get('applicant_credit_change',
0))
        respondent_credit_change =
int(data.get('respondent_credit_change', 0))

        # 解决仲裁
        success = arbitration_db.resolve_arbitration(
            aid=int(aid),
            admin_uid=admin_uid,
            decision=decision,
            penalty_amount=penalty_amount,
            penalized_user_uid=penalized_user_uid,
            applicant_credit_change=applicant_credit_change,
            respondent_credit_change=respondent_credit_change
        )

        if success:
            return JsonResponse({
                'status': 'success',
                'message': '仲裁解决成功',
                'data': {
                    'aid': aid,
                    'decision': decision,
                    'penalty_amount': penalty_amount
                }
            })
        else:
            return JsonResponse({
                'status': 'error',
                'message': '解决仲裁失败'
            }, status=500)
```

```
    except json.JSONDecodeError:
        return JsonResponse({
            'status': 'error',
            'message': '无效的JSON数据'
        }, status=400)
    except ValueError:
        return JsonResponse({
            'status': 'error',
            'message': '无效的仲裁ID或数据'
        }, status=400)
    except Exception as e:
        return JsonResponse({
            'status': 'error',
            'message': f'解决仲裁失败: {str(e)}'
        }, status=500)
```

三、系统各项功能数据库端操作主要代码与结果说明

对于各项系统核心业务功能，其原生SQL操作和对应的前端交互效果如下。

1. 用户注册与身份验证

功能描述： 处理用户注册时的唯一性检查和登录时的凭证校验。

数据库层原生SQL实现：

```
-- 用户注册：插入新用户记录
INSERT INTO users (username, email, password_hash,
    phone, coin, role, credit_score, avatar_url, student_id)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)

-- 根据ID获取用户
SELECT uid, username, email, phone, coin, role,
    credit_score, avatar_url, student_id, created_at, is_active
FROM users WHERE uid = %s

-- 用户登录验证：根据用户名或邮箱获取用户
SELECT uid, username, email, phone, coin, role, credit_score,
    avatar_url, student_id, created_at, is_active, password_hash
```

```
FROM users WHERE username = %s OR email = %s

-- 修改密码
UPDATE users SET password_hash = %s WHERE uid = %s AND is_active = 1
```

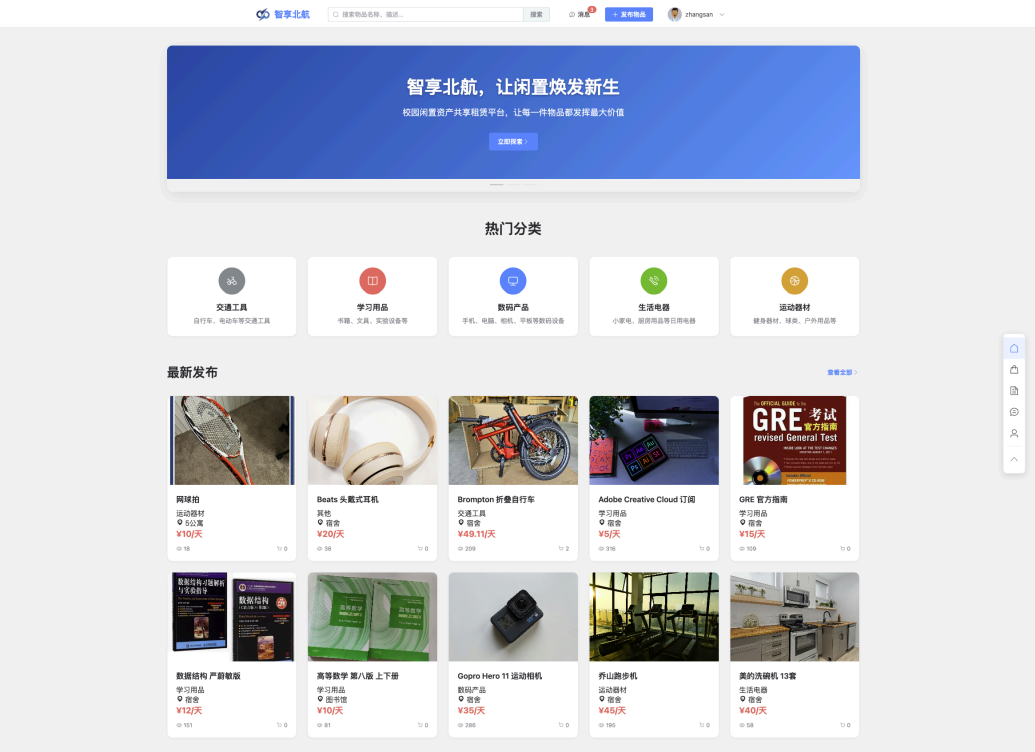
前端交互效果：

- **注册与登录页面**：收集用户的必要信息





- **首页**：展示横幅、热门分类与推荐物品



2. 物品发布与多维搜索

功能描述： 支持物主发布物品，并允许租客按分类、校区、关键词进行高效筛选。

数据库层原生SQL实现：

```
-- 物品发布
INSERT INTO items (owner_uid, cid, title, description, campus,
    location, daily_rent, deposit, image_urls)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)

-- 根据ID获取物品详情
SELECT i.*, c.cname as category_name, u.username as owner_name
FROM items i
LEFT JOIN categories c ON i.cid = c.cid
LEFT JOIN users u ON i.owner_uid = u.uid
WHERE i.iid = %s

-- 物品搜索和列表查询
SELECT i.*, c.cname as category_name, u.username as owner_name
FROM items i
LEFT JOIN categories c ON i.cid = c.cid
```

```
LEFT JOIN users u ON i.owner_uid = u.uid
{where_clause}
ORDER BY i.{sort_by} {sort_order}
LIMIT %s OFFSET %s

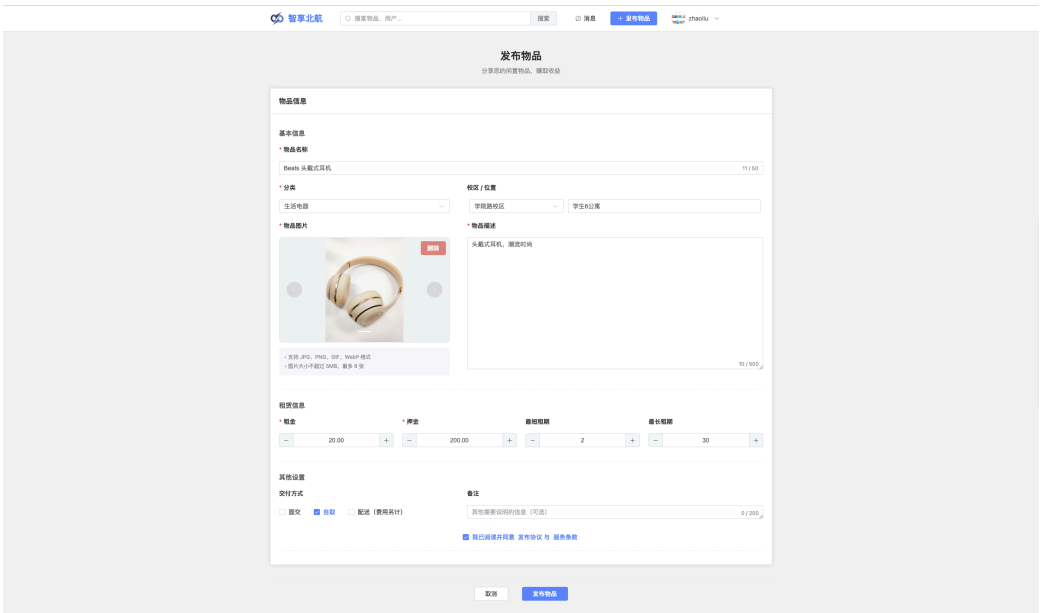
-- 物品搜索（关键词匹配）
WHERE (i.title LIKE %s OR i.description LIKE %s)

-- 增加浏览次数
UPDATE items SET view_count = view_count + 1 WHERE iid = %s

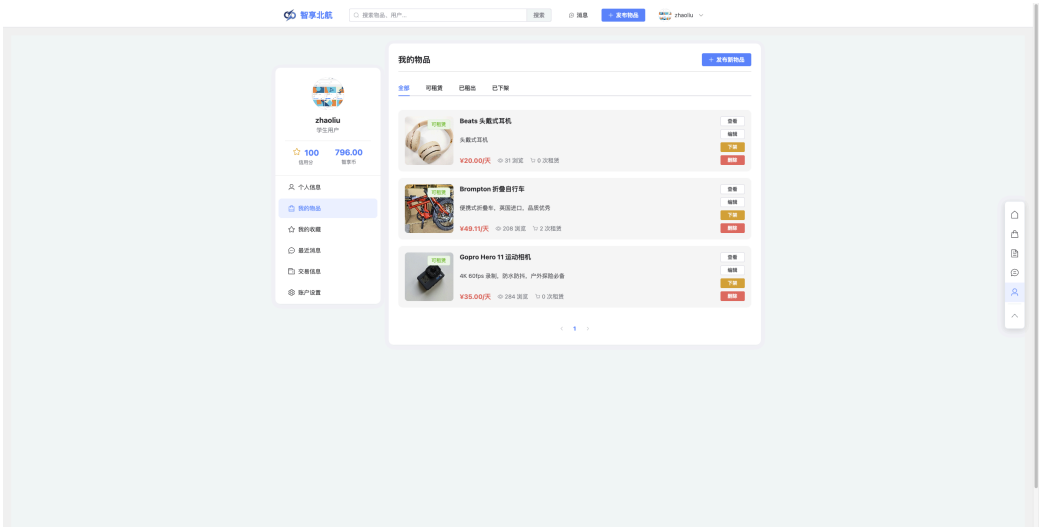
-- 软删除物品
UPDATE items SET status = 'removed' WHERE iid = %s
```

前端交互效果：

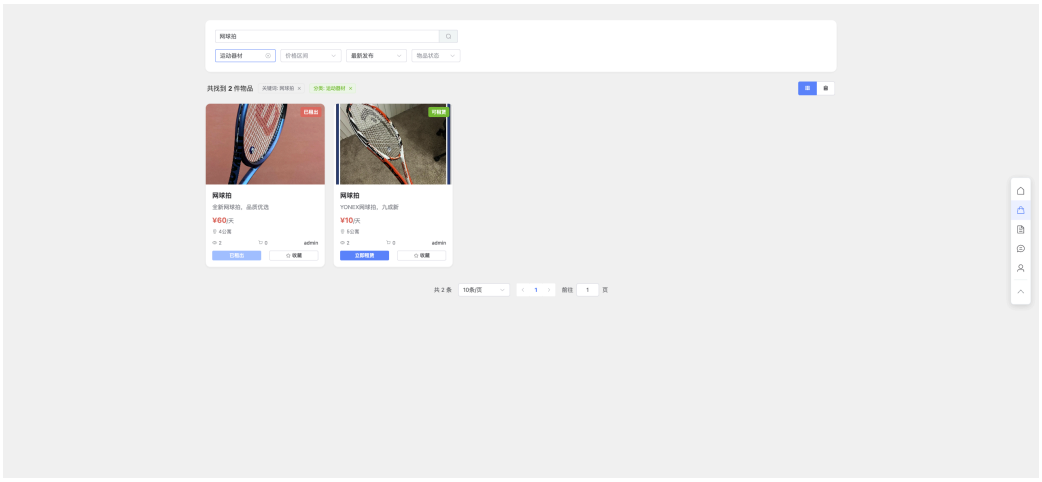
- **发布物品**：填写物品描述、价格、交付方式等信息



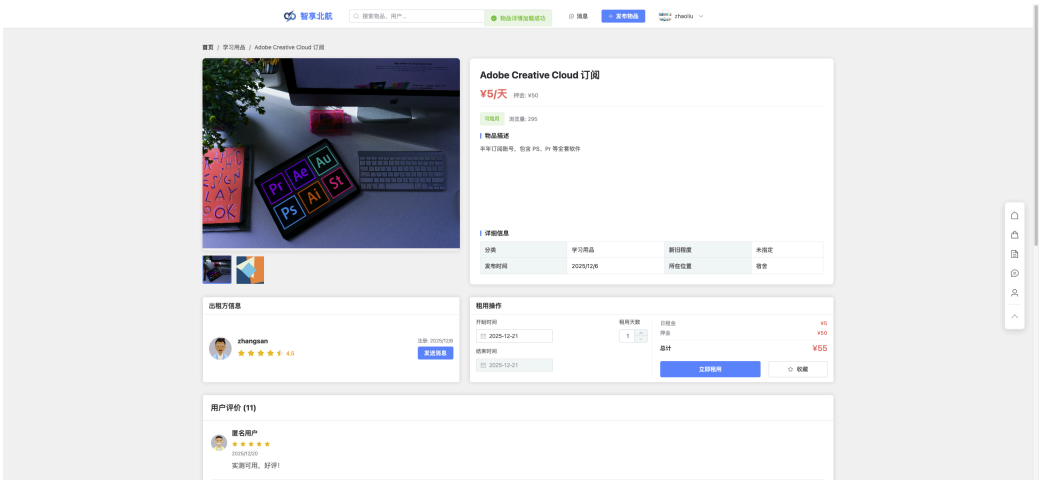
- **发布成功**：自动跳转至物品页面，显示物品状态为"可租借"，物主可以进行物品管理



- **搜索结果**：分页展示物品卡片，包含图片、价格、是否可租用等信息



- **物品详情**：详情页展示物品信息和评价，为感兴趣的用户提供租赁入口和出租方联系方式



3. 订单创建与状态管理

功能描述： 处理租赁订单的创建、时间冲突检测和状态流转管理。

数据库层原生SQL实现：

```
-- 创建订单
INSERT INTO orders (iid, renter_uid, start_time, end_time,
    daily_rent, deposit, total_rent, remark, status)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s, 'pending')

-- 获取订单详情
SELECT o.*,
       i.title as item_title, i.campus as item_campus,
            i.owner_uid as owner_uid,
       i.image_urls as item_image_urls,
       r.username as renter_name, r.phone as renter_phone,
       ow.username as owner_name, ow.phone as owner_phone
FROM orders o
LEFT JOIN items i ON o.iid = i.iid
LEFT JOIN users r ON o.renter_uid = r.uid
LEFT JOIN users ow ON i.owner_uid = ow.uid
WHERE o.oid = %s

-- 检查时间冲突
SELECT COUNT(*) as conflict_count
FROM orders
WHERE iid = %s
AND status NOT IN ('cancelled')
AND NOT (end_time <= %s OR start_time >= %s)

-- 更新订单状态
UPDATE orders SET status = %s WHERE oid = %s

-- 订单列表查询（支持分页和过滤）
SELECT o.*,
       i.title as item_title, i.campus as item_campus,
            i.owner_uid as owner_uid,
       i.image_urls as item_image_urls,
       r.username as renter_name,
       ow.username as owner_name
```

```
FROM orders o
LEFT JOIN items i ON o.iid = i.iid
LEFT JOIN users r ON o.renter_uid = r.uid
LEFT JOIN users ow ON i.owner_uid = ow.uid
{where_clause}
ORDER BY o.created_at DESC
LIMIT %s OFFSET %s
```

前端交互效果：

- **下单成功**：显示订单ID和状态"待物主确认"，自动扣除押金，显示成功提示



- **物主确认**：由物主再次确认，接受订单后才进入正式履约阶段



4. 信用分管理与仲裁处理

功能描述： 管理用户信用分变动和租赁纠纷仲裁。

数据库层原生SQL实现：

```sql
-- 创建信用变动记录
INSERT INTO credit_logs (uid, type, amount, reason,
    related_oid, admin_uid, before_score, after_score)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s)

-- 创建交易流水
INSERT INTO transactions (uid, role, direction, category,
    amount, related_oid, note, balance_after)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s)

-- 获取信用变动记录（包含用户和订单信息）
SELECT cl.*, u.username, o.oid as order_number,
    admin.username as admin_name
FROM credit_logs cl
LEFT JOIN users u ON cl.uid = u.uid
LEFT JOIN orders o ON cl.related_oid = o.oid
LEFT JOIN users admin ON cl.admin_uid = admin.uid
WHERE cl.id = %s

-- 更新用户信用分数
UPDATE users SET credit_score = %s WHERE uid = %s

-- 创建仲裁申请
INSERT INTO arbitrations (oid, applicant_uid, respondent_uid,
    admin_uid, dispute_reason, evidence_urls)
VALUES (%s, %s, %s, %s, %s, %s)

-- 获取仲裁详情
SELECT a.*,
       o.iid, o.start_time, o.end_time, o.total_rent, o.deposit,
       u1.username as applicant_name,
       u2.username as respondent_name,
       u3.username as admin_name
FROM arbitrations a
LEFT JOIN orders o ON a.oid = o.oid
```

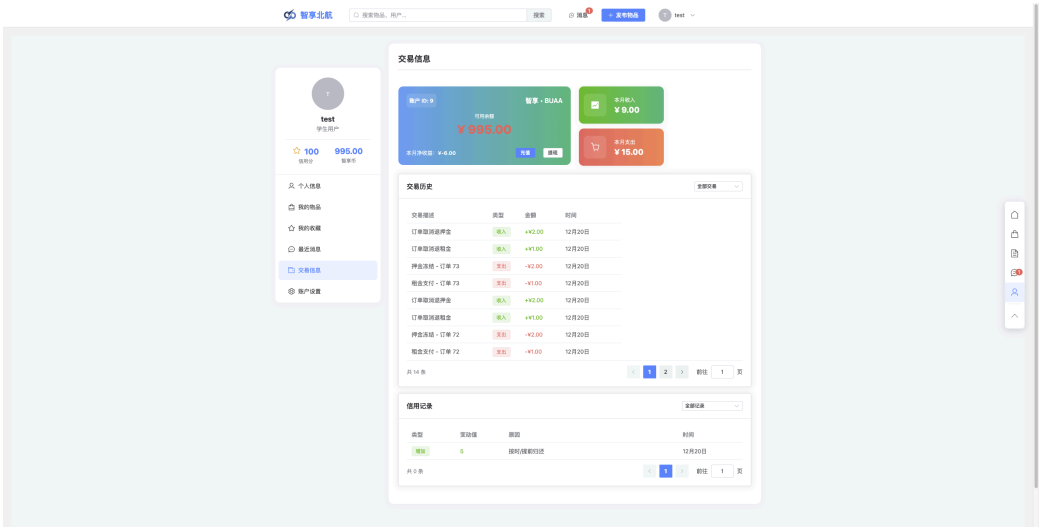
```sql
LEFT JOIN users u1 ON a.applicant_uid = u1.uid
LEFT JOIN users u2 ON a.respondent_uid = u2.uid
LEFT JOIN users u3 ON a.admin_uid = u3.uid
WHERE a.aid = %s

-- 更新仲裁状态
UPDATE arbitrations
SET status = 'resolved',
    admin_uid = %s,
    decision = %s,
    penalty_amount = %s,
    resolved_at = NOW()
WHERE aid = %s

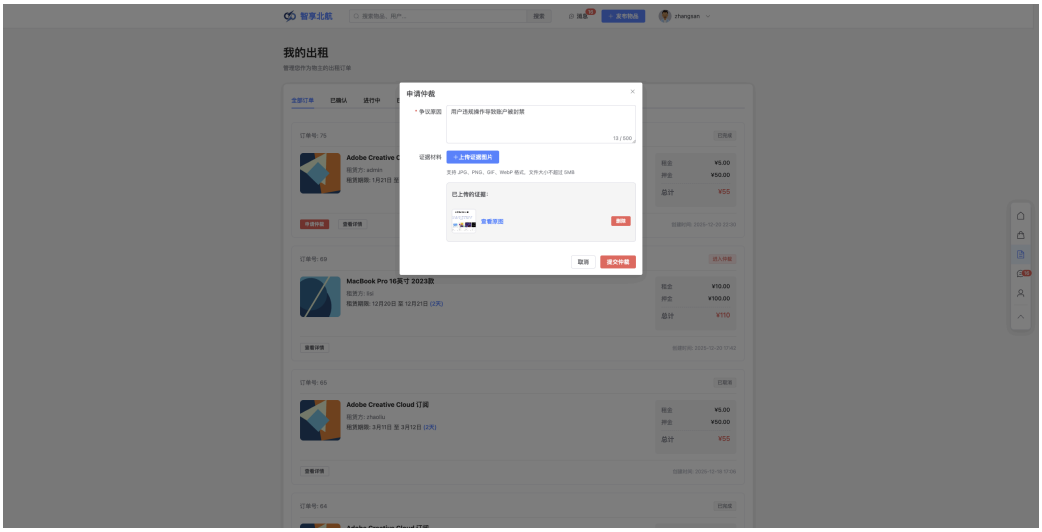
-- 信用统计查询
SELECT
    COUNT(*) as total_logs,
    SUM(CASE WHEN type = 'penalty' THEN 1 ELSE 0 END) as penalty_count,
    SUM(CASE WHEN type = 'reward' THEN 1 ELSE 0 END) as reward_count,
    SUM(CASE WHEN type = 'adjustment' THEN 1 ELSE 0 END) as
adjustment_count,
    SUM(CASE WHEN amount > 0 THEN amount ELSE 0 END) as total_gained,
    SUM(CASE WHEN amount < 0 THEN ABS(amount) ELSE 0 END) as total_lost,
    (SELECT credit_score FROM users WHERE uid = %s) as current_score
FROM credit_logs
WHERE uid = %s
```

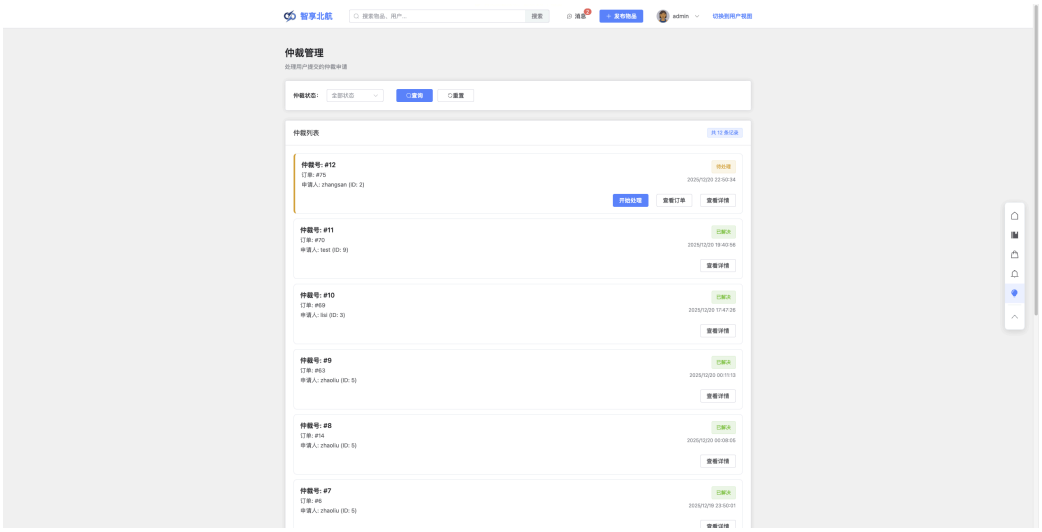
前端交互效果：

- **完成结算**：订单正常结束后显示扣款、信用分变化和余额变动，生成电子收据



- **仲裁申请**：发生争议时，用户可提交描述和证据图片，生成仲裁编号，显示申请状态





- **仲裁结果**：显示管理员裁决、罚金金额和信用分调整



5. 消息中心

功能描述： 支持用户间的消息通讯、系统通知推送、仲裁结果通知等，为平台用户提供完整的沟通交流功能。

数据库层原生SQL实现：

```sql
-- 创建用户消息记录
INSERT INTO messages (sender_uid, receiver_uid, content,
    message_type, is_read, created_at)
VALUES (%s, %s, %s, %s, 0, NOW())

-- 获取两个用户之间的对话历史
SELECT m.*,
       u1.username as sender_name, u1.avatar_url as sender_avatar,
       u2.username as receiver_name, u2.avatar_url as receiver_avatar
FROM messages m
LEFT JOIN users u1 ON m.sender_uid = u1.uid
LEFT JOIN users u2 ON m.receiver_uid = u2.uid
WHERE ((m.sender_uid = %s AND m.receiver_uid = %s) OR
       (m.sender_uid = %s AND m.receiver_uid = %s))
ORDER BY m.created_at ASC
LIMIT %s OFFSET %s

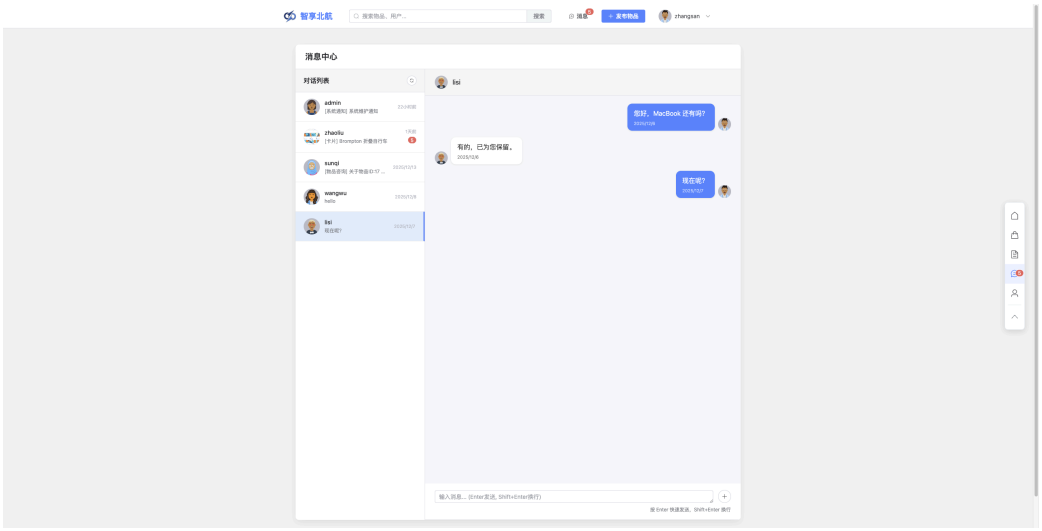
-- 标记消息为已读
UPDATE messages
```

```
SET is_read = 1, read_at = NOW()
WHERE sender_uid = %s AND receiver_uid = %s AND is_read = 0

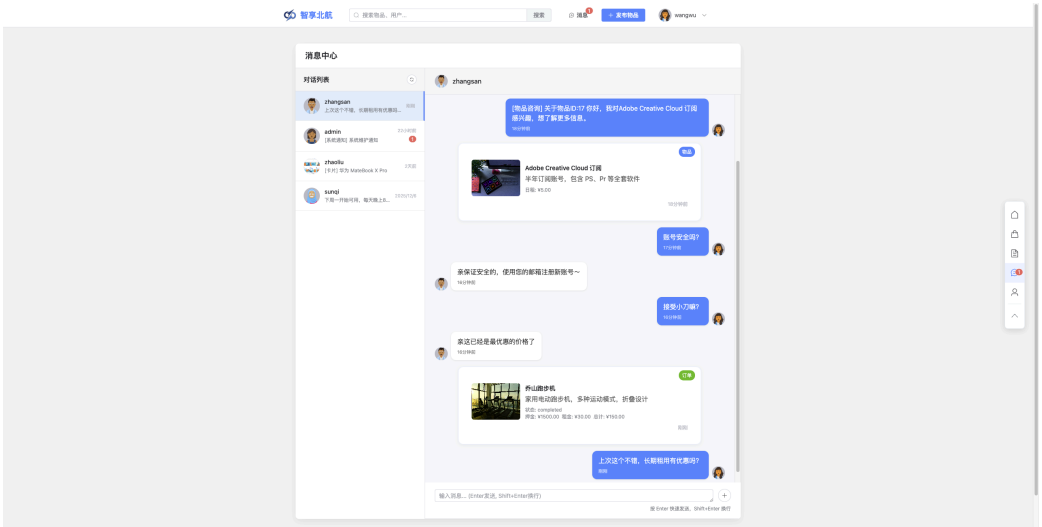
-- 获取用户未读消息统计
SELECT COUNT(*) as unread_count
FROM messages
WHERE receiver_uid = %s AND is_read = 0
```

前端交互效果：

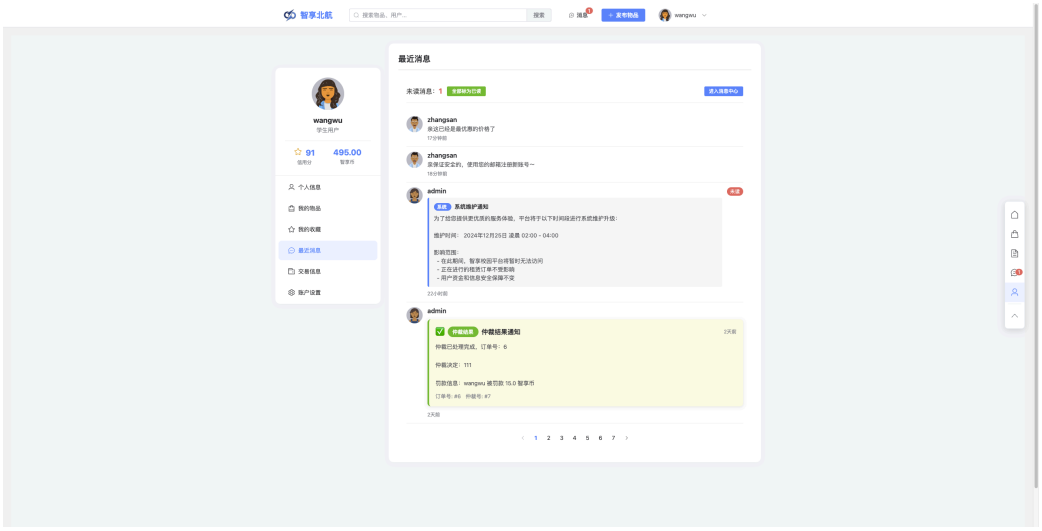
- **消息列表**：显示用户所有消息，未读消息使用红点标注数量



- **对话详情**：展示与特定用户的完整对话历史，支持发送文本、物品链接和双方历史订单的链接

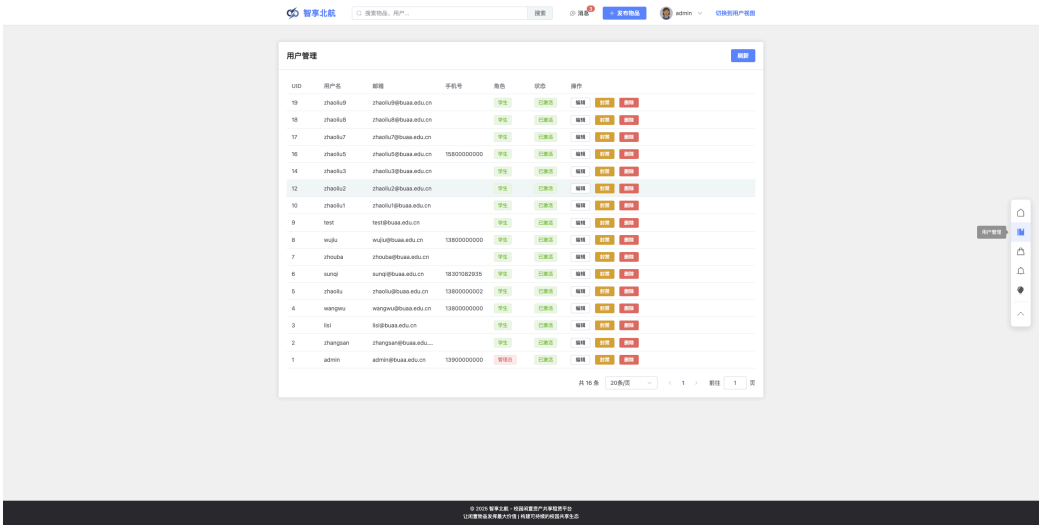


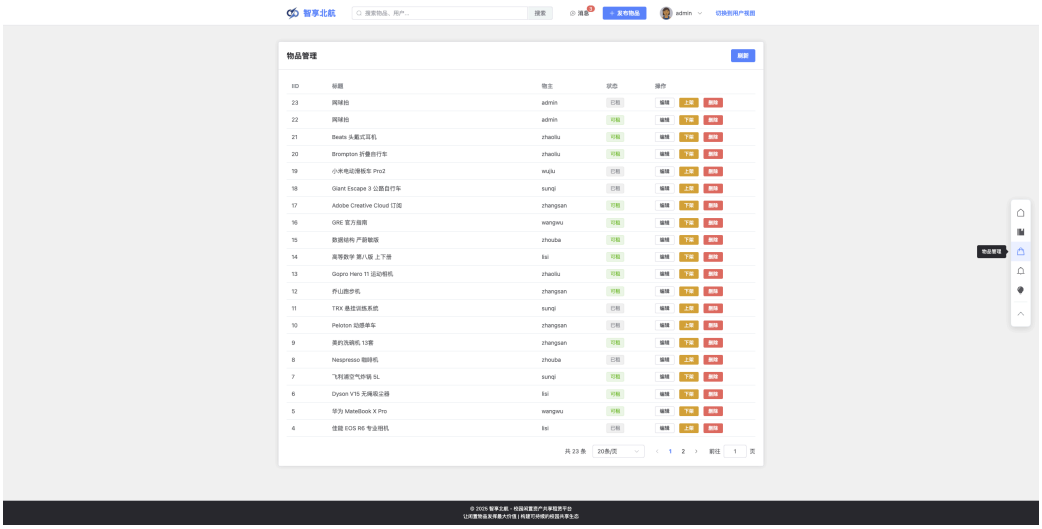
- **消息通知**：新消息提醒，显示未读消息数量，提供一键已读功能



6. 管理员后台操作

管理员具有在后台管理平台注册用户和已发布物品的权限，具体SQL实现类似前序章节呈现的代码，在此不再重复。





此外，管理员还可以发布系统通知，向所有用户推送重要信息、平台公告和维护通知。相较于普通信息，系统通知具有更加醒目的外观，以凸显其重要性。

数据库层原生SQL实现：

```sql
-- 获取所有普通用户（非管理员）用于系统通知
SELECT uid FROM users WHERE role != 'admin'

-- 创建系统通知消息记录
INSERT INTO messages (sender_uid, receiver_uid, content,
    message_type, is_read, created_at)
VALUES (%s, %s, %s, %s, 0, NOW())

-- 查询用户系统通知列表
SELECT m.*, u.username as sender_username
FROM messages m
LEFT JOIN users u ON m.sender_uid = u.uid
WHERE m.receiver_uid = %s
AND m.message_type = 'notification'
ORDER BY m.created_at DESC
LIMIT %s OFFSET %s
```